

1. Job Orchestration

1.1 Components

- **Job Runner (BullMQ Pro)**
 - Runs inside one or more worker services on DigitalOcean App Platform.
 - Uses Redis as the queue and persistence layer.
- **Workers**
 - Worker instances process jobs from Redis.
 - Multiple workers can run in parallel (scaled by App Platform).
- **Job Orchestration Logic**
 - Implemented in `job-runner.server.ts` and worker modules.
 - Manages job creation, dependencies, retries, and state transitions.

1.2 Job Types

- `import-products`
 - Bulk product and variant import from Shopify into PostgreSQL.
- `campaign-start`
 - Main campaign processing job:
 - Validations
 - Fetching products / bulk operations
 - Applying include/exclude rules
 - Analyzing collisions between campaigns
 - Preparing variant-level data
 - Sending mutations to Shopify to update prices
 - Final status updates
- `campaign-end`
 - Scheduled deactivation:
 - Revert or deactivate campaign
 - Revert prices where applicable
- `watch-stuck-jobs`
 - Monitoring job:
 - Checks for jobs stuck in `running` for too long
 - Performs recovery or raises alerts
- `migration`
 - Long-running database updates / data migrations

1.3 Dependencies

Job dependencies are modeled via the `JobDependency` table:

- **campaign-start depends on import-products**
 - Ensures product catalog is up-to-date before applying a campaign.
- **campaign-end depends on campaign-start**
 - Only runs after the campaign has successfully started.
- **watch-stuck-jobs monitors** both **campaign-start** and **campaign-end**.

The execution state machine is:

- **Scheduled** – planned for a future time.
 - **Pending** – waiting for dependencies to complete.
 - **Running** – currently executing on a worker.
 - **Completed** – finished successfully.
 - **Failed** – error occurred.
 - **Cancelled** – manually stopped by user.
 - **Stopped** – system-initiated halt (e.g. fatal error).
-

2. Job Execution Steps (Campaign Start)

The **campaign-start** job itself is multi-phase:

1. **validation (Step 1)**
 - Pre-flight checks:
 - Store/session validity
 - Campaign status & time window
 - Conflicting campaign rules
2. **fetchProducts (Step 2)**
 - Bulk API calls to Shopify to fetch product/variant data.
3. **importProducts (Step 3)**
 - Sync fetched data into PostgreSQL.
4. **handleIncludesExcludes (Step 4)**
 - Apply include filters and exclude rules to determine target variants.
5. **campaignAnalyze (Step 5)**
 - Collision detection & campaign priority rules between multiple campaigns.
6. **dataProcessing (Step 6)**
 - Prepare variant-level data, compute new prices, rounding, etc.
7. **writeMutations / workMutations (Step 7)**
 - Send actual Shopify mutations to update prices.
 - Handle rate limiting and retry logic.
8. **end (Step 8)**
 - Finalize status, write logs, emit completion events.

Throughout these steps:

- Updates are written to PostgreSQL (**Job**, **JobExecution**, campaign status)
 - WebSocket updates are published for progress visualization in the UI
 - Logger system and audit trail persist errors, warnings, and user actions
-

3. Current Problems & Limitations

3.1 Large Campaign Size

- Some campaigns target **2,000,000+ records** (products/variants) in a single operation.
- Currently, a **single campaign-start job** is responsible for processing the entire campaign from start to finish.
- This results in:
 - Extremely long-running jobs on a single worker.
 - High CPU and memory usage in one process.
 - Increased risk of:
 - Worker crashes
 - Timeouts
 - Node.js memory limits / GC pressure
 - Shopify API rate-limit issues

3.2 Worker Scaling Behavior on DigitalOcean App Platform

- Flashx uses **DigitalOcean App Platform Worker Services** to run BullMQ workers.
- Important behavior from DigitalOcean support:
 - Worker services **do not receive load-balanced traffic**.
 - The job system (BullMQ + our code) is solely responsible for distributing work.
 - When a large job is picked by **Worker-0**, it **stays on Worker-0** until completion.
 - If a second worker scales up, it will **remain idle** unless there are **other jobs** in the queue.

Effect in our system:

- A campaign with 200,000+ or 2,000,000+ variants is typically a **single heavy job**.
- That job is taken by one worker instance.
- When CPU on Worker-0 spikes to ~85–95%, DigitalOcean auto-scaler:
 - Detects high load on that instance and **starts Worker-1**.
 - But Worker-1 has nothing to do because the queue only has the one big job that is already in progress.
- Result:
 - **Uneven load distribution**: one worker is overloaded, others are idle.

- We see autoscaling events without actual throughput improvement.

DigitalOcean also explained that:

- Scale-up can be triggered even when average CPU seems lower (20–30%) due to short peaks over the threshold or memory/queue metrics.
- The platform **does not split a single heavy job** across multiple workers.
- The application must **break large workloads into smaller units** if we want parallelism.

3.3 Practical Symptoms

- Campaigns with millions of records:
 - Take very long time to complete.
 - Sometimes fail mid-way due to:
 - Timeouts
 - API throttling
 - Worker restarts or deployment cycles
 - Auto-scaling to multiple workers:
 - Does **not** improve performance for a single huge campaign.
 - Causes cost increase without matching performance gains.
 - When multiple big campaigns run concurrently:
 - Job queue can become backlogged.
 - Subsequent campaigns start late or stay in **pending** longer than expected.
-

4. Summary for System Engineer

- The current architecture centralizes a **very large workload** into a single **campaign-start** job.
- DigitalOcean's worker model **does not distribute work inside that job**; only our queue design can.
- For very large campaigns, the main bottleneck is:
 - One heavy job on one worker,
 - Combined with Shopify API limits and DB throughput.
- Any scaling or reliability work should focus on:
 - **Refactoring jobs into smaller units**
 - **Parallel processing across workers**
 - **Careful queue and resource tuning** to cooperate with Shopify and DigitalOcean constraints.